

Mock Design: Typeahead / Autocomplete System

Phase 4 · Interview-Ready · Estimated time: 1.5 hours

1. Concept From First Principles: Applying the Framework

Typeahead (search-as-you-type suggestions) is a favorite interview problem because it forces a genuinely interesting split: the core data structure question (how do you efficiently find all strings starting with a given prefix?) is classic computer science, while the system design question (how do you serve that lookup at low latency, at scale, with data that changes constantly?) is pure HLD. Strong answers handle both halves without letting either one crowd out the other.

Step 1 — Clarify requirements

- Functional: as a user types a prefix, return the top-K most relevant completions, ranked by some popularity/relevance signal, updated within tens of milliseconds of each keystroke.
- Non-functional: extremely low latency is the entire product experience — anything above ~100ms feels laggy to a typing user. Suggestions don't need to be perfectly real-time fresh; a few minutes of staleness on ranking is acceptable.

Step 2 — Estimate scale

Assume 500 million searches/day, and since a typeahead query fires on nearly every keystroke (not just on submit), the actual query volume is 5–10x that — call it 3–5 billion typeahead queries/day, roughly 40,000–60,000 queries/second average, several times that at peak. This volume, and the sub-100ms latency requirement, is exactly why this can't be a live database query per keystroke — it must be served from an in-memory, pre-built structure.

Step 3 — API

GET /suggest?prefix=swig -> ["swiggy", "swiggy instamart", "swiggy genie", ...], returning the top-K (typically 5–10) ranked completions for the given prefix.

Step 4 — High-level design

Two clearly separate paths, a distinction worth stating explicitly since it's the core architectural insight of this problem: a **data collection/ranking pipeline** (offline/near-real-time, built from search logs) that periodically builds the suggestion structure, and a **serving path** (fully online, must be fast) that answers user keystrokes against the already-built structure. Client → Load Balancer → Suggestion Service, which holds an in-memory **Trie** (prefix tree) mapping prefixes to ranked completions, sharded across multiple servers (e.g., by first 1–2 characters of the prefix, since that naturally bounds each shard's tree size) if the full trie doesn't fit one machine's RAM.

Step 5 — Deep dive: the Trie and keeping rankings fresh

A **Trie** is a tree where each node represents one character, and a path from root to node represents a prefix; each node also stores the top-K most popular completions for the prefix ending at that node, precomputed, so a lookup is just an $O(\text{prefix length})$ walk down the tree followed by an $O(1)$ read of the

precomputed top-K list — not a live re-ranking on every query, which is what makes sub-100ms latency achievable at this volume.

Building the Trie: an offline batch job (Phase 3, Ch 7's batch processing) periodically aggregates recent search query logs, counts frequency per query string, and rebuilds the Trie (or a delta-updates it) with fresh top-K completions per prefix node. This is deliberately **not real-time per search** — rebuilding the full structure every few minutes to hours is an acceptable trade given the stated non-functional requirement that slight ranking staleness is fine, and it dramatically simplifies the system versus trying to update rankings on every single search event.

A genuinely tricky follow-up: trending or breaking-news-style queries (a cricket match result, a sudden news event) need to surface faster than the normal batch cycle allows. A stronger answer names a **hybrid** approach: the batch-built Trie handles the steady-state long tail, while a separate, fast-updating real-time counter (using something like Phase 3 Ch 10's Count-Min Sketch for memory efficiency at high query cardinality) tracks short-term spikes and can boost or inject a trending query into results even before the next full batch rebuild.

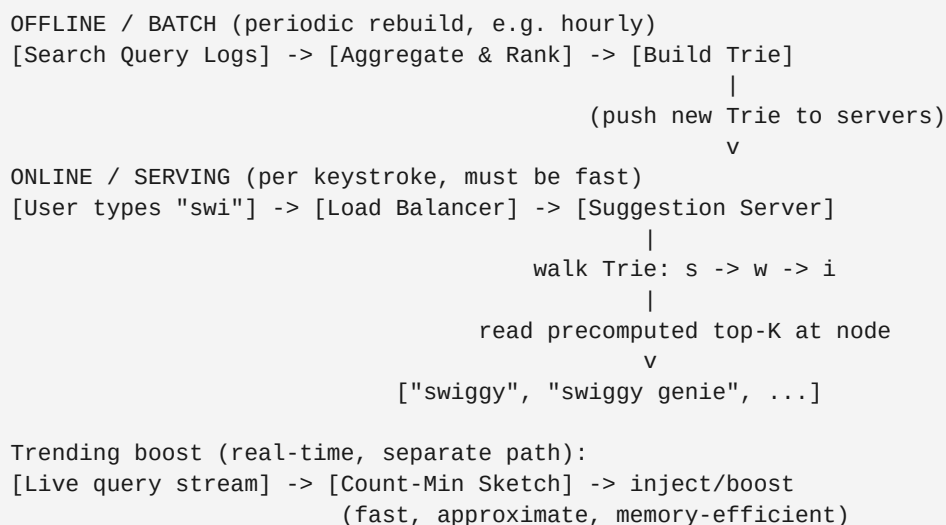
Step 6 — Trade-offs and wrap-up

Volunteer: sharding the Trie by prefix character range can create hot shards (many common English words start with a small set of letters) — discuss further sub-sharding of oversized branches, and note that the whole in-memory Trie fleet should itself sit behind a cache-friendly, replicated deployment for read availability, similar to Chapter 5's distributed cache design.

2. Real-World Example / Case Study

Google Search's autocomplete and Flipkart's product-search typeahead are both real production instances of this exact split between an offline ranking pipeline and a fast in-memory serving path — the Trie structure and its precomputed top-K per node is the standard, widely-taught approach specifically because it converts a potentially expensive live ranking computation into a cheap tree walk at serve time.

3. Diagram



4. Trade-offs Table

Decision	Alternative	Why this choice
Trie with precomputed top-K per node	Live ranking query per keystroke	O(prefix length) lookup vs. expensive live computation; meets sub-100ms need
Batch rebuild (minutes-to-hours)	Real-time update per search event	Acceptable staleness trade-off; far simpler than continuous re-ranking
Hybrid: batch Trie + real-time trend counter	Batch-only	Surfaces breaking/trending queries faster than the normal rebuild cycle
Shard Trie by prefix character range	Single unsharded Trie	Distributes memory/load; requires handling hot-shard imbalance

5. Common Interview Questions

Q1: "Why not just query the database directly for matching strings on every keystroke?"

Model approach: State the latency math directly — tens of thousands of queries/second at sub-100ms requirement rules out a live database scan; an in-memory Trie with precomputed rankings is what makes the latency target achievable.

Q2: "How do you keep suggestions fresh without rebuilding the entire Trie on every search?"

Model approach: Describe the offline batch rebuild cycle explicitly, and name the hybrid real-time trend counter as the mechanism for surfacing genuinely time-sensitive spikes faster than the batch cycle allows.

Q3: "How would you shard the Trie if it doesn't fit on one machine?"

Model approach: Shard by prefix character range (e.g., a-h, i-p, q-z), and proactively name the resulting hot-shard risk from uneven prefix popularity, with further sub-sharding of oversized branches as the mitigation.

6. Practice Exercises

- 1 Run the full six-step framework for "design a typeahead suggestion system," out loud, with a timer.
- 2 Implement a basic Trie insert and prefix-lookup function in pseudocode, including a top-K completions list per node.
- 3 Design the hybrid real-time trending-query boost mechanism in more detail: what data structure, what refresh interval, how does it merge with the batch-built results?

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Design Search Autocomplete System - System Design Interview"	Covers the Trie-based approach and the offline/online split directly.

Language	Video & Channel	Why it helps
English	"Design Autocomplete or Typeahead Suggestions Front-end Mock Interview Chakde System Design Ep. 7"	Mock interview format featuring an India-based (Flipkart) engineer's perspective.
Hindi	"System Design Google Autocomplete Typeahead Suggestion HLD Auto Suggestion TRIE Data Structure"	Hindi/Hinglish walkthrough centered specifically on the Trie data structure for this problem.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Bilingual course covering typeahead as part of the broader curriculum.

8. Key Takeaways

- Typeahead splits cleanly into an offline ranking/build pipeline and a fast, purely-lookup online serving path — keep both halves distinct in your answer.
- A Trie with precomputed top-K completions per node turns an expensive live ranking problem into a cheap $O(\text{prefix length})$ tree walk.
- Batch rebuilding (minutes to hours) is an acceptable staleness trade-off given the stated non-functional requirements.
- A hybrid real-time trend counter (e.g., Count-Min Sketch) handles breaking/trending queries the batch cycle alone would miss.
- Sharding the Trie by prefix range introduces a real hot-shard risk from uneven natural language prefix popularity.
- Sub-100ms latency is the actual product requirement driving nearly every design decision in this system.
- This problem rewards balancing the data-structure question (the Trie) and the systems question (freshness, scale, sharding) rather than over-focusing on either alone.